

AWS Cloud Monitoring System

MOHAMMED NASSER MOHAMMED NAJI

220218440

EC2

Dashboard

EC2 Global View

Events

Instances

Instances

Instance Types

Launch Templates

Instances (1/1) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

All states

Instance state : running

Clear filters

< 1 >

☒	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
☒		i-0f4f0a9224892978e	Running	t3.micro	3/3 checks passed	View alarms	us-east-1f

Problem

Monitoring cloud servers manually is inefficient and unreliable. Without an automated monitoring system, high CPU usage on cloud servers can go unnoticed, which may lead to performance degradation, service downtime, or increased operational costs. Additionally, users may not receive timely notifications when system resources exceed safe limits. Therefore, there is a need for an automated cloud monitoring solution that can track resource usage and send alerts in real time.

Alarms by AWS service [info](#)

Services

■ In alarm 0 ■ Insufficient data 0 ■ OK 1

EC2

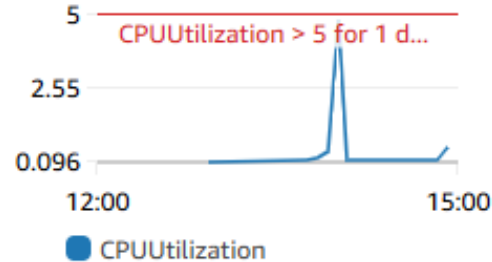


Recent alarms [info](#)

[View recent alarms dashboard](#)

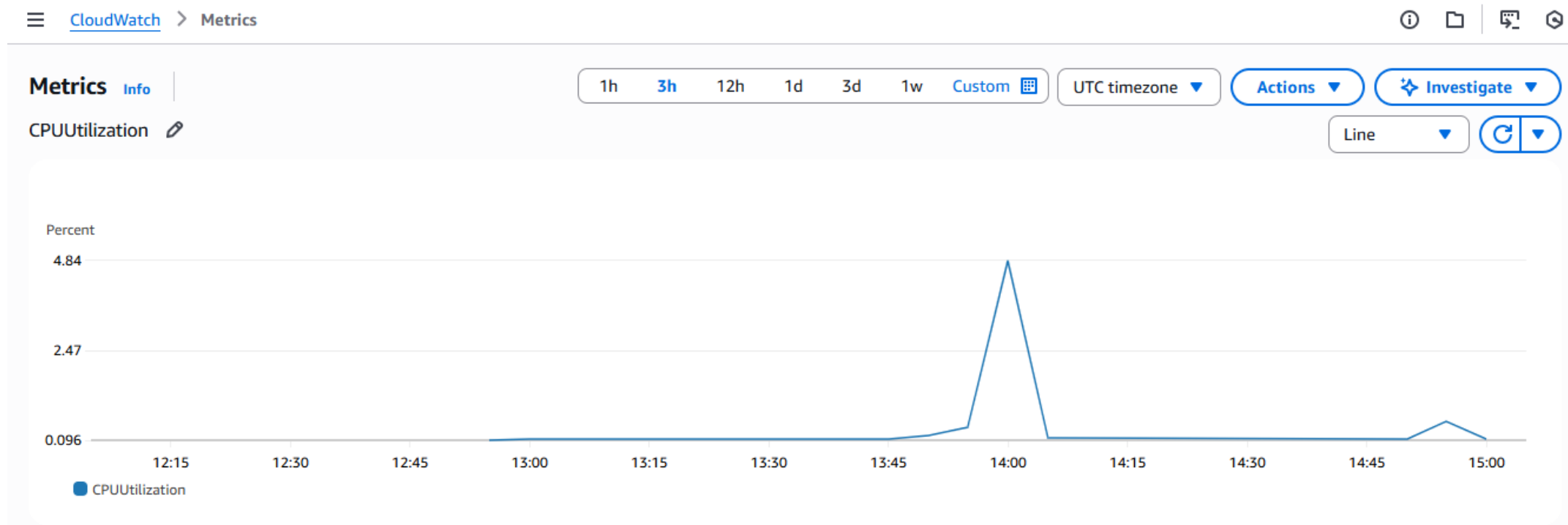
✓ High-CPU-Alarm (i) ⋮

Percent



Solution

The solution is to implement an automated cloud monitoring •
instance is monitored 2 system using AWS services. An EC
using Amazon CloudWatch to track CPU utilization in real time.
A CloudWatch alarm is configured to trigger when CPU usage
exceeds a predefined threshold. When the alarm is activated,
Amazon SNS sends an automatic email notification to alert the
user. This solution provides real-time monitoring, automation,
and timely alerts, ensuring better performance and system
.reliability



Key Features

- Automated CloudWatch alarms for high CPU usage
- CPU utilization2 Real-time monitoring of EC
- Email notifications using Amazon SNS
- Cloud-based solution using AWS services
- Automated alerting without manual intervention
- instance after testing2 Cost-efficient by stopping the EC

Alarms (1)

☐ Hide Auto Scaling alarms

Clear selection

Create composite alarm

Actions ▾

Create alarm

Alarm state: Any ▾

Alarm type: Any ▾

Actions status: Any ▾

< 1 >

☐	Name ▾	State ▾	Last state update (UTC) ▾	Conditions	Acti
☐	High-CPU-Alarm	✓ OK	2026-01-06 14:05:53	CPUUtilization > 5 for 1 datapoints within 5 minutes	✓

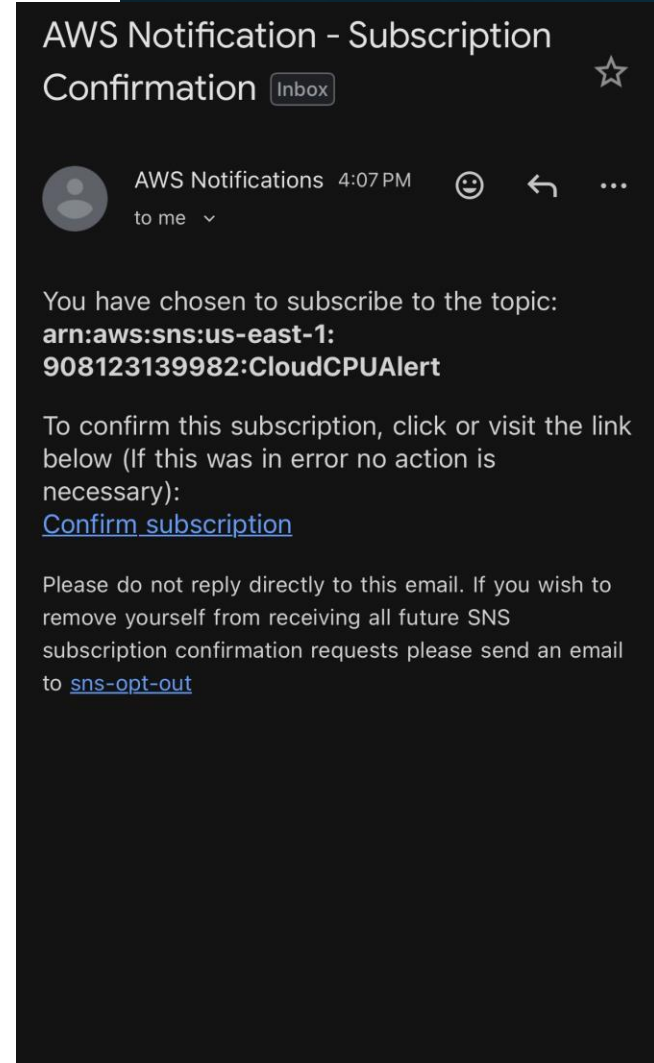
Technical Architecture

The system is built using multiple AWS services working together to provide

- instance generates 2 automated monitoring and alerting. An Amazon EC performance metrics such as CPU utilization. These metrics are collected by Amazon CloudWatch, which continuously monitors the instance. A CloudWatch alarm evaluates the metrics and triggers an alert when the CPU usage exceeds a predefined threshold. Once triggered, the alarm sends a .notification through Amazon SNS, which delivers an email alert to the user

Code Logic (How It Works)

- The system works based on event-driven automation. Amazon CloudWatch continuously monitors the CPU utilization of instance. When the CPU usage 2 the EC exceeds the defined threshold, a CloudWatch .alarm is triggered
- Once the alarm is triggered, an event is generated and sent to AWS services. Amazon SNS receives this event and sends an email notification to the subscribed user. The code logic ensures that alerts are sent automatically .without any manual intervention



Roadmap & Future Improvements

Monitor additional metrics such as memory usage, disk usage, and network traffic

Support multiple EC2 instances instead of a single instance

Add SMS or mobile push notifications in addition to email alerts

Integrate AWS Auto Scaling to automatically handle high resource usage

Create a CloudWatch dashboard for better visualization

Improve alert logic using different thresholds and anomaly detection

Results of the Project

- The AWS Cloud Monitoring System was implemented and tested successfully. The EC2 instance was monitored in real time using Amazon CloudWatch, which continuously tracked CPU utilization.
- During testing, high CPU usage was generated on the EC2 instance. When the CPU utilization exceeded the defined threshold, the CloudWatch alarm changed its state to **ALARM**.
- After the CPU usage decreased, the alarm returned to the **OK** state.
- Overall, the project achieved its objectives by providing automated monitoring, real-time alerts, and reliable notification delivery using AWS services.

